Retrace: Database Design Document

Version 0.1 | Based on system design v0.3 and PRD v0.6 | 21 October 2026 | Draft for review

This document defines every database, schema, table, column, key, and index in Retrace, along with the roles that may touch them. It follows the decisions in the system design: a central control database, one database per tenant, one schema per service inside each tenant database, and one database role per service per tenant.

1. How to read this document

- Section 2 lists the conventions that apply to every table, so they are not repeated.
- Section 3 describes the databases, schemas, roles, and the three common tables every service has (jobs, outbox, inbox).
- Section 4 covers the control database. Sections 5 to 8 cover the tenant schemas: core, ingestion, ai, and notifications.
- Section 9 lists every reference that crosses a schema boundary (these are plain ids, with no foreign keys).
- Section 10 covers retention and deletion. Section 11 covers tables for later phases. Section 12 lists open items and corrections needed in the system design.
- Phase notes: tables are built in the phase of the service that owns them (Core in phase 1, Ingestion and AI in phase 2, Notifications in phase 3).

2. Conventions

Topic	Rule
Primary keys	Every table has id uuid as the primary key, a UUIDv7 generated by the application. Tables with a composite key say so.

Topic	Rule
Standard columns	Every table also has created_at timestamptz not null default now() and updated_at timestamptz not null default now(), unless the table is append-only (then updated_at is omitted and the table says so). These columns are not repeated in the table listings.
Time	All timestamps are timestamptz and stored in UTC. Date-only values use date.
Names	snake_case. Table names are plural. Foreign key columns are named after the table they point to, with _id.
Enumerations	Stored as text with a check constraint listing the allowed values, not as database enum types, because check constraints are easier to change in a migration.
Flexible data	jsonb for data that varies in shape (plans, scores, claims). Anything filtered or joined on its own is a real column.
Foreign keys	Real foreign keys inside a schema. Across schemas there are no foreign keys, only plain uuid columns (see section 9).
Users in tenant data	A user is referenced by user_id (a plain uuid pointing to the control database) plus a display-name snapshot (user_name) on rows where history matters (comments, approvals, versions, audit log, actions). Reports then still read correctly after a person leaves.
Deletion	Tenant data is hard-deleted. Users and memberships are deactivated, not deleted. Deleting an organization drops its database.
Secrets	Secret values are stored as ciphertext (bytea) with a nonce and a key_version that points to the tenant data key. Plain secrets are never stored.

Topic	Rule
Text limits	Free text columns have sensible length limits enforced by the application. Search text uses PostgreSQL full-text search.
Demo data	The organization row carries is_demo. Demo data lives only in a tenant marked as demo, so it never mixes with real tenants.
Index names	ix_table_columns for ordinary indexes, ux_table_columns for unique indexes.

3. Databases, schemas, and roles

3.1 Topology

Database	Owner service	Contents
control (one)	Core	Users, organizations, memberships, invitations, and the tenant registry
tenant_{org_id} (one per organization)	All services	Four schemas, one per service: core, ingestion, ai, notifications. A realtime schema is added in a later phase.

3.2 Roles and grants

Role	Purpose	Permissions
control_core	The Core service on the control database	Read and write on all control tables. No other service connects to the control database.
{service}_{org_id} (one per service per tenant)	A service working inside one tenant	Usage and all privileges on its own schema only. No access to other schemas. The public schema is revoked.

Role	Purpose	Permissions
migrator	The migration runner only	Owns the schemas and applies migrations. Services never use it at runtime.
readonly_ops (optional)	Support and diagnostics	Select on selected tables, with approval and logging. Never given secret tables.

Special grants:

- core.audit_log: the core role gets INSERT and SELECT only, with no UPDATE or DELETE. A trigger also rejects updates and deletes as a second guard.
- Secret tables (ingestion.integration_secrets and the key columns of ai.model_settings): only the owning service role can read them, and the contents are ciphertext.
- Each role has its search_path set to its own schema.

3.3 Common tables (every tenant schema has all three)

jobs

Tracks long-running work started by a user or the system, so the interface can show progress.

Column	Type	Notes
kind	text not null	For example discover_evidence, generate_draft, render_export
status	text not null	queued, running, succeeded, failed, cancelled
requested_by_user_id	uuid null	Reference to control.users
entity_type	text null	The kind of thing the job works on
entity_id	uuid null	The id of that thing
progress	smallint not null default 0	0 to 100

Column	Type	Notes
attempts	int not null default 0	
result	jsonb null	Small result summary
error_message	text null	User-safe message only. Details go to logs.
started_at, finished_at	timestamptz null	

Indexes: ix_jobs_status_created (status, created_at), ix_jobs_entity (entity_type, entity_id).

outbox

Events written in the same transaction as the data change, then published by a relay. Only the publishing fields are ever updated.

Column	Type	Notes
event_type	text not null	For example postmortem.approved
aggregate_type	text not null	
aggregate_id	uuid not null	
payload	jsonb not null	Ids and small facts only, never large content or secrets
occurred_at	timestamptz not null default now()	
published_at	timestamptz null	Null until the relay publishes it
attempts	int not null default 0	
last_error	text null	

Indexes: ix_outbox_unpublished (occurred_at) where published_at is null.

inbox

Records events already handled, so repeats are ignored. Composite primary key (event_id, consumer). No id column.

Column	Type	Notes
event_id	uuid not null	The id of the event
consumer	text not null	The handler name
event_type	text not null	
received_at	timestamptz not null default now()	
processed_at	timestamptz null	

4. Control database

Owned by Core. It holds identity and the tenant registry. No business data lives here.

users

Column	Type	Notes
email	citext not null	Unique (ux_users_email)
email_verified_at	timestamptz null	
password_hash	text null	Null for users who only sign in with OAuth
display_name	text not null	
avatar_url	text null	
status	text not null default 'active'	active, deactivated
last_login_at	timestamptz null	
failed_login_count	int not null default 0	Reset on success
locked_until	timestamptz null	Temporary lockout after repeated failures

oauth_accounts

Column	Type	Notes
user_id	uuid not null	FK to users, on delete cascade
provider	text not null	google, github
provider_user_id	text not null	
email_at_link	citext null	

Unique: (provider, provider_user_id).

email_tokens

Column	Type	Notes
user_id	uuid not null	FK to users
purpose	text not null	verify_email, password_reset
token_hash	bytea not null	Only the hash is stored. Unique.
expires_at	timestamptz not null	
used_at	timestamptz null	

refresh_tokens

Column	Type	Notes
user_id	uuid not null	FK to users
org_id	uuid null	FK to organizations. The organization last used with this token.
family_id	uuid not null	All tokens from one sign-in share a family, for reuse detection
token_hash	bytea not null	Unique
issued_at	timestamptz not null	
expires_at	timestamptz not null	

Column	Type	Notes
rotated_at	timestamptz null	Set when exchanged for a new token
revoked_at	timestamptz null	
user_agent	text null	
ip_address	inet null	

If a rotated token is presented again, the whole family is revoked. Indexes: ix_refresh_tokens_user, ix_refresh_tokens_family.

organizations

The id of an organization is also its tenant id.

Column	Type	Notes
name	text not null	
slug	text not null	Unique, used in URLs
status	text not null	provisioning, active, suspended, deleting
is_demo	boolean not null default false	
created_by	uuid not null	FK to users

memberships

Column	Type	Notes
org_id	uuid not null	FK to organizations
user_id	uuid not null	FK to users
role	text not null	admin, member, manager, viewer. Admin and member are built first.
status	text not null default 'active'	active, removed
invited_by	uuid null	FK to users

Column	Type	Notes
joined_at	timestamptz not null	
removed_at	timestamptz null	

Unique: (org_id, user_id). Rule: every active organization must keep at least one active admin. The service enforces it, and a deferred constraint trigger backs it up. Indexes: ix_memberships_user (user_id, status).

invitations

Column	Type	Notes
org_id	uuid not null	FK to organizations
email	citext not null	
role	text not null	
token_hash	bytea not null	Unique
invited_by	uuid not null	FK to users
expires_at	timestamptz not null	
accepted_at	timestamptz null	
accepted_by	uuid null	FK to users
revoked_at	timestamptz null	

Unique partial index: one open invitation per (org_id, email) where accepted_at and revoked_at are null.

db_servers

The PostgreSQL servers that can host tenant databases. This lets the registry move tenants between servers.

Column	Type	Notes
name	text not null	Unique
host	text not null	

Column	Type	Notes
port	int not null default 5432	
region	text null	
status	text not null	open, full, draining
max_tenants	int not null	Placement limit

tenant_registry

One row per organization (a one-to-one relation). Services resolve tenants through Core using this table.

Column	Type	Notes
org_id	uuid not null	FK to organizations, unique
db_server_id	uuid not null	FK to db_servers
database_name	text not null	Unique per server
status	text not null	provisioning, active, read_only, deleting, failed
schema_versions	jsonb not null default '{}'	Migration version per service, for example core at 0007
provisioned_at	timestamptz null	
last_error	text null	

tenant_service_roles

References to the database role of each service. Passwords are not stored here, only the name of the secret in Key Vault.

Column	Type	Notes
org_id	uuid not null	FK to organizations
service	text not null	core, ingestion, ai, notifications, realtime
role_name	text not null	

Column	Type	Notes
key_vault_secret_name	text not null	
status	text not null	active, rotating, revoked
rotated_at	timestamptz null	

Unique: (org_id, service).

tenant_keys

Column	Type	Notes
org_id	uuid not null	FK to organizations
version	int not null	Increases when the data key is rotated
wrapped_key	bytea not null	The data key, encrypted by the Key Vault master key
master_key_id	text not null	Which master key version wrapped it
status	text not null	active, retired

Unique: (org_id, version). Secret rows in tenant databases store the key_version they used.

provisioning_runs

Column	Type	Notes
org_id	uuid not null	FK to organizations
kind	text not null	provision, migrate, move, delete
status	text not null	queued, running, succeeded, failed
step	text null	The step reached, so a retry can resume
attempt	int not null default 1	
started_at, finished_at	timestamptz null	
error	text null	

jwt_signing_keys

Column	Type	Notes
kid	text not null	Key id placed in tokens. Unique.
public_key_pem	text not null	Published to the gateway and services
private_key_ref	text not null	Name of the Key Vault secret. The private key is never stored here.
status	text not null	active, retiring, retired
not_before, not_after	timestamptz not null	Validity window, for rotation

security_events

Append-only record of user-level security events that do not belong to one tenant. No updated_at.

Column	Type	Notes
user_id	uuid null	FK to users (null for failed sign-ins with an unknown email)
event_type	text not null	login_success, login_failed, password_reset_requested, password_changed, oauth_linked, refresh_reuse_detected, account_locked
ip_address	inet null	
user_agent	text null	
details	jsonb null	No secrets

Indexes: ix_security_events_user (user_id, created_at).

5. Tenant schema: core

Owned by the Core service and used through the role core_{org_id}. The common tables jobs, outbox, and inbox from section 3.3 also exist in this schema.

org_settings

One row per tenant that holds organization-level settings.

Column	Type	Notes
org_id	uuid not null	Reference to control.organizations (no foreign key)
stall_days	int not null default 7	Check: greater than 0. Days without an update before an action item counts as stalled
escalation_days	int not null default 3	Days an item stays stalled before escalation
pre_due_days	int not null default 2	Days before the due date for the first reminder
evidence_retention_days	int null	Null means keep evidence until the incident is deleted
timezone	text not null default 'UTC'	Display only. Storage stays in UTC.
enabled_optional_roles	text[] not null default '{}'	Values: manager, viewer (later phase)
singleton	boolean not null default true	Unique and check (singleton), so the table can hold only one row

incidents

Column	Type	Notes
source	text not null	pagerduty, manual
external_id	text null	PagerDuty incident id
title	text not null	
description	text null	The author's short description of what happened
description_submitted_at	timestamptz null	Starts evidence discovery

Column	Type	Notes
incident_type	text null	outage, performance, security, data, other
severity	text null	sev1, sev2, sev3, sev4
status	text not null default 'open'	open, resolved, postmortem_in_progress, closed
started_at	timestamptz null	First symptom, when known
detected_at	timestamptz null	First alert
acknowledged_at	timestamptz null	
mitigated_at	timestamptz null	
resolved_at	timestamptz null	Check: not before started_at when both are set
created_by_user_id	uuid null	Reference to control.users
created_by_name	text null	Snapshot

These time fields feed the recovery-time analytics (time to detect, acknowledge, mitigate, resolve). Indexes: ux_incidents_source_external (source, external_id) where external_id is not null, ix_incidents_status (status, severity, started_at desc), ix_incidents_started (started_at desc).

incident_services

The services an incident affected. Composite primary key (incident_id, service_id), no id column.

Column	Type	Notes
incident_id	uuid not null	FK to incidents, on delete cascade
service_id	uuid not null	Reference to ingestion.services (no foreign key)
service_name	text not null	Snapshot, so history survives a renamed service

incident_responders

Column	Type	Notes
incident_id	uuid not null	FK to incidents, on delete cascade
user_id	uuid null	Reference to control.users, when the responder is a Retrace user
display_name	text not null	
external_handle	text null	For example a Slack or PagerDuty handle
incident_role	text null	commander, responder, observer
source	text not null	pagerduty, slack, manual

Index: ix_incident_responders_incident (incident_id).

postmortems

One postmortem per incident. This table holds the current working version.

Column	Type	Notes
incident_id	uuid not null	FK to incidents, unique, on delete cascade
title	text not null	
status	text not null default 'draft'	draft, in_review, changes_requested, approved, published
content	jsonb not null	The document as ProseMirror JSON
content_text	text not null default ''	Plain text derived from content, for search and the blameless check
search_vector	tsvector	Generated from title and content_text
revision	int not null default 1	Increases on every save. Used to detect a stale save (for example, a second browser tab)
current_version_no	int not null default 0	The latest named version

Column	Type	Notes
author_user_id	uuid not null	Reference to control.users
author_name	text not null	Snapshot
root_cause_category	text null	Used by repeat-incident detection (later phase)
root_cause_text	text null	Extracted from the root-cause section
generation_status	text not null default 'none'	none, running, failed
blameless_checked_revision	int null	The revision on which the blameless check last ran
blameless_open_count	int not null default 0	Open findings on the latest check
approved_version_id	uuid null	FK to postmortem_versions. The version that was approved.
approved_at	timestamptz null	
published_at	timestamptz null	
is_locked	boolean not null default false	True when approved or published, so the document is read-only until reopened

Checks: published requires approved. Indexes: ux_postmortems_incident (incident_id), ix_postmortems_status (status), ix_postmortems_search (GIN on search_vector).

postmortem_versions

Full snapshots taken at defined events. Autosave only updates the current row in postmortems.

Column	Type	Notes
postmortem_id	uuid not null	FK to postmortems, on delete cascade
version_no	int not null	Increases from 1

Column	Type	Notes
trigger	text not null	generation, submit, request_changes, approve, publish, reopen, restore, manual
label	text null	A short readable label
content	jsonb not null	The document snapshot
content_text	text not null	
created_by_user_id	uuid null	Null for system-created versions
created_by_name	text null	Snapshot
draft_run_id	uuid null	Reference to ai.draft_runs for AI-generated versions (no foreign key)
model_name	text null	The model that produced it, if any
prompt_version	text null	
blameless_open_count	int null	Open findings at that time

Unique: (postmortem_id, version_no). Append-only, so there is no updated_at.

comment_threads

Column	Type	Notes
postmortem_id	uuid not null	FK to postmortems, on delete cascade
anchor_mark_id	uuid not null	The id stored in the comment mark inside the document
quoted_text	text not null	The text the comment was attached to when created
status	text not null default 'open'	open, resolved, orphaned (the anchored text was removed)
created_by_user_id	uuid not null	
created_by_name	text not null	
resolved_by_user_id	uuid null	

Column	Type	Notes
resolved_by_name	text null	
resolved_at	timestamptz null	

Unique: (postmortem_id, anchor_mark_id). Index: ix_comment_threads_postmortem (postmortem_id, status).

comments

Column	Type	Notes
thread_id	uuid not null	FK to comment_threads, on delete cascade
body	text not null	
author_user_id	uuid not null	
author_name	text not null	Snapshot
edited_at	timestamptz null	

Index: ix_comments_thread (thread_id, created_at).

approvals

Append-only record of review decisions. No updated_at.

Column	Type	Notes
postmortem_id	uuid not null	FK to postmortems, on delete cascade
version_id	uuid not null	FK to postmortem_versions. The exact version decided on.
decision	text not null	approved, changes_requested
note	text null	Required when decision is changes_requested
decided_by_user_id	uuid not null	
decided_by_name	text not null	Snapshot

Column	Type	Notes
decided_by_role	text not null	The role at the time of the decision
decided_at	timestamptz not null default now()	

Check: the decider is not the postmortem author (enforced in the service, with a trigger as a backstop). Index: ix_approvals_postmortem (postmortem_id, decided_at).

action_items

Column	Type	Notes
incident_id	uuid not null	FK to incidents, on delete cascade
postmortem_id	uuid null	FK to postmortems, on delete set null
title	text not null	
description	text null	
source	text not null	ai_suggested, manual
ai_confidence	numeric(4,3) null	For suggested items
owner_user_id	uuid null	Reference to control.users
owner_name	text null	Snapshot
due_date	date null	
priority	text not null default 'medium'	low, medium, high, critical
status	text not null default 'open'	open, in_progress, blocked, done, dropped
drop_reason	text null	Required when dropped
success_criterion	text null	
verification_note	text null	
verification_url	text null	

Column	Type	Notes
confirmed_at	timestamptz null	Set when a person confirms the owner and due date. Nothing is tracked or reminded until then.
stalled_since	timestamptz null	Set by the daily scan, cleared on recovery
last_status_change_at	timestamptz not null default now()	
completed_at	timestamptz null	Check: set when status is done
external_issue_url	text null	For a linked GitHub issue (later)
created_by_user_id	uuid null	

Checks: confirmed requires owner and due date, dropped requires drop_reason. Indexes: ix_action_items_status_due (status, due_date), ix_action_items_owner (owner_user_id, status), ix_action_items_stalled (stalled_since) where stalled_since is not null.

action_item_events

Append-only change history. No updated_at.

Column	Type	Notes
action_item_id	uuid not null	FK to action_items, on delete cascade
event_type	text not null	created, status_changed, owner_changed, due_changed, verified, confirmed
from_value	text null	
to_value	text null	
actor_user_id	uuid null	Null for system actions
actor_name	text null	

Index: ix_action_item_events_item (action_item_id, created_at).

exports

Column	Type	Notes
kind	text not null	postmortem_pdf, postmortem_markdown, audit_report (later)
incident_id	uuid not null	FK to incidents, on delete cascade
postmortem_id	uuid null	FK to postmortems, on delete cascade
version_id	uuid null	FK to postmortem_versions. The version exported.
status	text not null	queued, running, ready, failed
blob_path	text null	Path in Azure Blob Storage
size_bytes	bigint null	
sha256	bytea null	Checksum of the file
requested_by_user_id	uuid not null	
expires_at	timestamptz null	When the file is removed from storage
error_message	text null	User-safe

analytics_summaries

Precomputed figures for dashboards, refreshed by a job.

Column	Type	Notes
period_start	date not null	
granularity	text not null	week, month
service_id	uuid null	Null means all services (reference to ingestion.services)
severity	text null	Null means all severities
incident_count	int not null	
mean_time_to_detect_s	int null	Seconds

Column	Type	Notes
mean_time_to_acknowledge_s	int null	
mean_time_to_mitigate_s	int null	
mean_time_to_resolve_s	int null	
action_items_total	int not null default 0	
action_items_done	int not null default 0	
action_items_stalled	int not null default 0	
computed_at	timestamptz not null	

Unique index on (period_start, granularity, coalesce(service_id, zero uuid), coalesce(severity, empty text)), so each combination has one row.

audit_log

Append-only and tamper-evident. No updated_at. The core role has INSERT and SELECT only.

Column	Type	Notes
seq	bigint not null	Gap-free sequence per tenant, unique. Fixes the order of the chain.
occurred_at	timestamptz not null default now()	
actor_type	text not null	user, service, system
actor_user_id	uuid null	
actor_name	text null	Snapshot
actor_role	text null	Role at the time
source_service	text not null	core, ingestion, ai, notifications

Column	Type	Notes
action	text not null	For example postmortem.approved, integration.connected, member.role_changed
entity_type	text null	
entity_id	uuid null	
summary	text not null	One readable line
details	jsonb null	Relevant values, never secrets or evidence text
request_id	text null	Trace id from the gateway
prev_hash	bytea not null	The entry_hash of the previous entry. The first entry uses 32 zero bytes.
entry_hash	bytea not null	SHA-256 over prev_hash and the canonical form of this entry

How the chain works:

1. Appending takes a per-tenant advisory lock, reads the last row, assigns seq and prev_hash, computes entry_hash, and inserts. The lock keeps the chain linear when two writes happen at once.
2. Other services publish audit.recorded events. Core consumes them and appends them through the same code path, so there is one writer.
3. A verify job recomputes every hash in order and reports the first mismatch. Editing or deleting any row breaks every hash after it.
4. An audit-ready report (later phase) includes the current head hash and the verification result.

Indexes: ux_audit_log_seq (seq), ix_audit_log_action (action, occurred_at), ix_audit_log_entity (entity_type, entity_id), ix_audit_log_actor (actor_user_id, occurred_at).

6. Tenant schema: ingestion

Owned by the Ingestion service and used through the role ingestion_{org_id}. The common tables jobs, outbox, and inbox exist here too. Evidence is stored as redacted text only. Original payloads are not kept, so the source link is the way back to the original.

integration_connections

Column	Type	Notes
provider	text not null	slack, github, pagerduty
status	text not null default 'active'	active, error, revoked
auth_mode	text not null default 'platform_app'	platform_app, org_app (the organization's own Slack app, to avoid distribution rate limits)
display_name	text not null	Workspace, organization, or account name
external_account_id	text not null	Slack team id, GitHub installation id, PagerDuty account id
scopes	text[] not null default '{}'	Granted scopes, for review
webhook_token_hash	bytea null	Hash of the token in the PagerDuty webhook URL. Unique where not null.
connected_by_user_id	uuid not null	Reference to control.users
connected_by_name	text not null	Snapshot
connected_at	timestamptz not null default now()	
last_sync_at	timestamptz null	
last_error	text null	User-safe
health_checked_at	timestamptz null	

Unique: (provider, external_account_id).

integration_secrets

Ciphertext only. Only the ingestion role can read this table.

Column	Type	Notes
connection_id	uuid not null	FK to integration_connections, on delete cascade
kind	text not null	access_token, refresh_token, client_id, client_secret, webhook_secret, app_private_key
ciphertext	bytea not null	Encrypted with the tenant data key
nonce	bytea not null	
key_version	int not null	The tenant_keys version used
rotated_at	timestamptz null	

Unique: (connection_id, kind).

allowlist_entries

The only places the AI and discovery are allowed to read.

Column	Type	Notes
connection_id	uuid not null	FK to integration_connections, on delete cascade
resource_type	text not null	slack_channel, github_repo, pagerduty_service
external_id	text not null	
display_name	text not null	
is_private	boolean not null default false	Private channels are excluded unless an Admin adds them on purpose
added_by_user_id	uuid not null	
added_by_name	text not null	

Unique: (connection_id, resource_type, external_id).

services

The service catalog.

Column	Type	Notes
name	text not null	Unique among non-archived services
description	text null	
owning_team	text null	
archived_at	timestamptz null	Archived services stay for history but are not offered for new incidents
created_by_user_id	uuid null	

service_links

Column	Type	Notes
service_id	uuid not null	FK to services, on delete cascade
link_type	text not null	slack_channel, github_repo, pagerduty_service, runbook_url
allowlist_entry_id	uuid null	FK to allowlist_entries, on delete set null
external_id	text not null	
display_name	text null	

Unique: (service_id, link_type, external_id). Links to sources must exist in the allowlist before discovery can use them.

discovery_runs

Column	Type	Notes
incident_id	uuid not null	Reference to core.incidents (no foreign key)
status	text not null	queued, running, awaiting_confirmation, confirmed, failed

Column	Type	Notes
window_start	timestamptz not null	
window_end	timestamptz not null	
plan	jsonb not null	The search plan: sources, channels, repositories, terms
stats	jsonb not null default '{}'	Candidates found, included, and excluded per source, and any rate-limit waits
timeline_revision	int not null default 0	Increases each time the timeline is rebuilt for this incident
started_at, finished_at	timestamptz null	
confirmed_by_user_id	uuid null	
confirmed_at	timestamptz null	
error_message	text null	User-safe

Index: ix_discovery_runs_incident (incident_id, created_at desc).

evidence_items

Column	Type	Notes
incident_id	uuid not null	Reference to core.incidents (no foreign key)
discovery_run_id	uuid null	FK to discovery_runs, on delete set null
source	text not null	slack, github, pagerduty, manual
source_ref	text not null	Stable id from the source, used for deduplication
kind	text not null	message, alert, deploy, pull_request, commit, note
url	text null	Link back to the original
actor_display	text null	Name or handle shown with the item

Column	Type	Notes
occurred_at	timestamptz not null	
text_redacted	text not null	Redacted text only
redaction_count	int not null default 0	
thread_ref	text null	Groups messages in the same thread
relevance_score	numeric(4,3) null	0 to 1
score_reasons	jsonb null	Why it was scored that way, for the preview
included	boolean not null default false	
added_manually	boolean not null default false	

Unique: (incident_id, source, source_ref). Indexes: ix_evidence_incident_time (incident_id, occurred_at), ix_evidence_incident_included (incident_id, included).

timeline_events

Column	Type	Notes
incident_id	uuid not null	Reference to core.incidents (no foreign key)
occurred_at	timestamptz not null	
sort_key	int not null	Position among events, from time and source priority
summary	text not null	One line
detail	text null	
source	text not null	The main source: slack, github, pagerduty, manual
key_moment	text null	first_symptom, detection, escalation, mitigation, resolution

Column	Type	Notes
phase	text null	detection, diagnosis, mitigation, recovery, other
has_discrepancy	boolean not null default false	Sources disagree on the time
discrepancy_note	text null	
is_manual	boolean not null default false	Added or edited by a person. Kept when the timeline is rebuilt.
is_hidden	boolean not null default false	
revision	int not null	The timeline_revision that produced it
created_by_user_id	uuid null	

Index: ix_timeline_incident_order (incident_id, sort_key). Rebuilding replaces automatic events and keeps manual ones.

timeline_event_evidence

Links each timeline event to the evidence behind it. Composite primary key (timeline_event_id, evidence_item_id).

Column	Type	Notes
timeline_event_id	uuid not null	FK to timeline_events, on delete cascade
evidence_item_id	uuid not null	FK to evidence_items, on delete cascade

This link is what makes a citation in a postmortem traceable to its sources.

7. Tenant schema: ai

Owned by the AI service and used through the role ai_{org_id}. The common tables jobs, outbox, and inbox exist here too.

model_settings

The required organization model setting. One row per tenant.

Column	Type	Notes
mode	text not null default 'default'	default (the Retrace default model) or custom
provider	text null	azure_openai, openai, anthropic, openai_compatible. Null in default mode.
model_name	text null	
endpoint_url	text null	
api_key_ciphertext	bytea null	Encrypted with the tenant data key
api_key_nonce	bytea null	
key_version	int null	
api_key_hint	text null	Last four characters, for display
max_context_tokens	int null	Used to decide how finely to split input
supports_embeddings	boolean not null default false	
embedding_model	text null	
last_tested_at	timestamptz null	
last_test_ok	boolean null	
updated_by_user_id	uuid null	
updated_by_name	text null	
singleton	boolean not null default true	Unique and check (singleton), so the table holds one row

Check: in custom mode, provider, model_name, endpoint_url, and api_key_ciphertext are required.

usage_counters

Column	Type	Notes
usage_date	date not null	
model_mode	text not null	default, custom
operation	text not null	draft, section, blameless, inline_edit
calls	int not null default 0	
input_tokens	bigint not null default 0	
output_tokens	bigint not null default 0	

Unique: (usage_date, model_mode, operation). The daily limit on the default model is checked against the sum of calls for the day.

draft_runs

Column	Type	Notes
incident_id	uuid not null	Reference to core.incidents
postmortem_id	uuid null	Reference to core.postmortems
requested_by_user_id	uuid not null	
status	text not null	queued, running, succeeded, partial, failed
provider	text not null	
model_name	text not null	
prompt_version	text not null	
timeline_revision	int not null	The timeline the draft was built from
phase_summaries	jsonb null	Summaries per phase, with event ids
resulting_version_no	int null	The version created in Core
claims_total	int not null default 0	

Column	Type	Notes
claims_downgraded	int not null default 0	Claims turned into hypotheses by validation
input_tokens	bigint not null default 0	
output_tokens	bigint not null default 0	
started_at, finished_at	timestamptz null	
error_message	text null	User-safe

Index: ix_draft_runs_incident (incident_id, created_at desc).

draft_section_runs

Column	Type	Notes
draft_run_id	uuid not null	FK to draft_runs, on delete cascade
section_key	text not null	summary, impact, detection, response, root_cause, contributing_factors, went_well, went_poorly, lessons, action_items
attempt	int not null default 1	A retry after a failed validation is attempt 2
status	text not null	succeeded, failed
output	jsonb null	Text and claims. Each claim has a statement, a kind (fact or hypothesis), and event ids.
validation_errors	jsonb null	
input_tokens	int not null default 0	
output_tokens	int not null default 0	

Unique: (draft_run_id, section_key, attempt).

blameless_findings

Column	Type	Notes
postmortem_id	uuid not null	Reference to core.postmortems
checked_revision	int not null	The document revision the check ran on
run_id	uuid not null	Groups the findings of one check
rule_group	text not null	attribution, judgment, counterfactual, absolute
matched_text	text not null	
anchor	jsonb not null	Where the text sits in the document, so the editor can highlight it
reason	text not null	Short explanation
suggested_rewrite	text null	Null if the model was unavailable
status	text not null default 'open'	open, accepted, edited, dismissed, stale
resolved_by_user_id	uuid null	
resolved_at	timestamptz null	

Index: ix_blameless_postmortem_status (postmortem_id, status). Findings tied to older revisions become stale when the text changes.

inline_edit_events

Append-only record of inline AI actions. No updated_at.

Column	Type	Notes
postmortem_id	uuid not null	
user_id	uuid not null	
action	text not null	blameless, clearer, shorter, expand_with_evidence, custom
selection_chars	int not null	Length only. The text is not stored.

Column	Type	Notes
outcome	text not null	streamed, accepted, rejected, failed
input_tokens	int not null default 0	
output_tokens	int not null default 0	

8. Tenant schema: notifications

Owned by the Notifications service and used through the role notifications_{org_id}. The common tables jobs, outbox, and inbox exist here too. This service keeps no copy of action item data. It asks Core for items each time.

reminder_log

Prevents duplicate reminders.

Column	Type	Notes
action_item_id	uuid not null	Reference to core.action_items (no foreign key)
stage	text not null	pre_due, stalled, escalated
recipient_user_id	uuid not null	
channel	text not null	email, in_app
notification_id	uuid null	FK to notifications, on delete set null
sent_at	timestamptz not null default now()	

Unique: (action_item_id, stage, recipient_user_id, channel).

notifications

Column	Type	Notes
recipient_user_id	uuid not null	

Column	Type	Notes
type	text not null	draft_ready, review_requested, approved, changes_requested, action_due, action_stalled, action_escalated
title	text not null	
body	text null	
entity_type	text null	
entity_id	uuid null	
link_path	text null	Path inside the app
read_at	timestamptz null	

Index: ix_notifications_recipient (recipient_user_id, read_at, created_at desc).

notification_settings

Column	Type	Notes
user_id	uuid not null	Unique
email_enabled	boolean not null default true	
in_app_enabled	boolean not null default true	
muted_types	text[] not null default '{}'	Notification types the user turned off

email_log

Column	Type	Notes
recipient_email	citext not null	Personal data. Removed with the retention rules in section 10.
template	text not null	
subject	text not null	
status	text not null	queued, sent, failed, bounced
provider_message_id	text null	

Column	Type	Notes
attempts	int not null default 0	
sent_at	timestamptz null	
error_message	text null	
notification_id	uuid null	FK to notifications, on delete set null

Index: ix_email_log_status (status, created_at).

9. References across schemas and databases

No foreign keys cross a schema or database boundary. These columns hold plain uuids. Integrity is kept by services checking ids through each other's APIs when writing, by events that clean up related data, and by a periodic reconciliation job that reports orphans.

From	To	Notes
User id columns in every tenant schema (created_by_user_id, owner_user_id, actor_user_id, and similar)	control.users.id	Display names are copied as snapshots so history stays readable
core.org_settings.org_id	control.organizations.id	
core.incident_services.service_id	ingestion.services.id	Service name is copied as a snapshot
core.analytics_summaries.service_id	ingestion.services.id	
core.postmortem_versions.draft_run_id	ai.draft_runs.id	
ingestion.discovery_runs.incident_id, ingestion.evidence_items.incident_id, ingestion.timeline_events.incident_id	core.incidents.id	Cleaned up by the incident.deleted event

From	To	Notes
ai.draft_runs.incident_id and postmortem_id	core.incidents.id and core.postmortems.id	Cleaned up by the incident.deleted event
ai.blameless_findings.postmortem_id, ai.inline_edit_events.postmortem_id	core.postmortems.id	Cleaned up by the incident.deleted event
notifications.reminder_log.action_item_id	core.action_items.id	
notifications.notifications.entity_id	Varies by entity_type (mostly core tables)	
Service role references in control.tenant_service_roles	Roles in each tenant database	Created and removed by the provisioning job

10. Retention and deletion

Data	Retention	How it is removed
Postmortems, versions, comments, approvals	Until the incident is deleted	Incident deletion cascades inside the core schema
Evidence items and timeline events	Until the incident is deleted, or until the organization evidence_retention_days passes	The incident.deleted event, or the purge job
Draft runs, blameless findings, inline edit events	Until the incident is deleted	The incident.deleted event. Usage counters are kept, since they hold no content.
Action items and their events	Until the incident is deleted	Cascade in core. Notifications cleans up related reminders and notifications.

Data	Retention	How it is removed
Audit log	For the life of the tenant, and never edited	Dropped with the tenant database. Entries about a deleted incident stay, because they hold no evidence text.
Exports	Until expires_at	The purge job deletes the Blob file and the row
Jobs	30 days after finishing	Purge job
Outbox rows	7 days after publishing	Purge job
Inbox rows	30 days	Purge job
Notifications	90 days after being read	Purge job
Email log	90 days	Purge job, because it holds email addresses
Email tokens	7 days after expiry	Purge job
Refresh tokens	30 days after expiry or revocation	Purge job
Security events	1 year, configurable	Purge job
Users	Kept, deactivated on request. A deletion request replaces email and display name with placeholders in the control database.	Tenant snapshots keep the old name until an Admin runs a scrub (see open items)
An entire organization	Until the Admin deletes it	The deletion flow in the system design: mark deleting, stop access, drop the database and roles, remove keys, secrets, and Blob files, remove the registry entry

Backups: Azure Database for PostgreSQL backs up the whole server, not individual databases. Deleted data therefore remains in server backups until they expire. For per-tenant restore and export, a nightly logical backup of each tenant database to Blob

Storage is planned (see open items).

11. Tables for later phases

These are designed now so earlier tables do not need to change when they arrive.

core.repeat_links (repeat-incident detection)

Column	Type	Notes
incident_id	uuid not null	FK to incidents, on delete cascade
repeat_of_incident_id	uuid not null	FK to incidents, on delete cascade
category_match	boolean not null	
text_similarity	numeric(4,3) not null	
score	numeric(4,3) not null	0.4 times category match plus 0.6 times text similarity
status	text not null default 'proposed'	proposed, confirmed, dismissed
decided_by_user_id	uuid null	
decided_by_name	text null	
decided_at	timestamptz null	

Unique: (incident_id, repeat_of_incident_id). Check: the two ids differ.

ai.postmortem_embeddings (only if an embedding model is configured)

Column	Type	Notes
postmortem_id	uuid not null	Reference to core.postmortems
model_name	text not null	
embedding	vector(n) not null	The dimension n is fixed when the feature is enabled, because it depends on the model

Index: an approximate nearest-neighbor index (HNSW) on embedding.

realtime schema (real-time co-editing)

Table	Columns	Notes
doc_updates	id bigint identity, postmortem_id uuid, update bytea, created_at	Append-only live document updates
doc_snapshots	postmortem_id uuid (primary key), snapshot bytea, state_vector bytea, last_update_id bigint, created_at	Compacted state. Content is written back to core.postmortems at snapshot events.

The optional Manager and Viewer roles need no new tables, because memberships.role already allows them.

12. Summary, performance notes, and open items

12.1 Table count

Location	Tables
Control database	14: users, oauth_accounts, email_tokens, refresh_tokens, organizations, memberships, invitations, db_servers, tenant_registry, tenant_service_roles, tenant_keys, provisioning_runs, jwt_signing_keys, security_events
core schema	14 plus 3 common: org_settings, incidents, incident_services, incident_responders, postmortems, postmortem_versions, comment_threads, comments, approvals, action_items, action_item_events, exports, analytics_summaries, audit_log
ingestion schema	9 plus 3 common: integration_connections, integration_secrets, allowlist_entries, services, service_links, discovery_runs, evidence_items, timeline_events, timeline_event_evidence

Location	Tables
ai schema	6 plus 3 common: model_settings, usage_counters, draft_runs, draft_section_runs, blameless_findings, inline_edit_events
notifications schema	4 plus 3 common: reminder_log, notifications, notification_settings, email_log

That is 14 control tables and 45 tables in every tenant database, 59 in total (not counting later phases).

12.2 Performance notes

- The busiest tables per tenant are evidence_items, timeline_events, and audit_log. Their indexes lead with incident_id or time, matching how they are read.
- Search uses a GIN index on postmortems.search_vector. No other search index is needed in v1.
- Partitioning is not used in v1. If a tenant grows large, audit_log and evidence_items are the candidates, partitioned by month.
- Because each tenant has its own database and each service its own role, connection counts grow with tenants times services. Pools stay very small and a pooler sits in front of the server.

12.3 Open items and changes needed elsewhere

1. **System design event catalog.** Add an incident.deleted event (published by Core, consumed by Ingestion, AI, and Notifications) so related data is cleaned up.
2. **Per-tenant backups.** Azure backs up the server as a whole. Decide on a nightly logical backup (pg_dump to Blob Storage) of each tenant database, which also supports tenant export and moving a tenant.
3. **Name scrub.** Decide whether an Admin tool replaces a removed user's name snapshots in tenant data when a deletion request is made.
4. **Embedding dimension.** Fixed when embeddings are enabled, because the vector column needs one size.
5. **Document size limits.** Set a maximum size for postmortem content (for example 2 MB of JSON) and a limit on manual versions, since each version is a full snapshot.

6. **Citation storage.** Citations live inside the ProseMirror JSON as nodes that carry timeline event ids. If citation queries are needed (for example, which postmortems cite a given event), a derived citations table can be added later.
7. **Graphviz.** The next deliverable is Graphviz code in six diagrams: the control database, the core schema, the ingestion schema, the ai schema, the notifications schema, and an overview of how services reference each other by id.